

Design Assignment
CSA1610 – DATA WAREHOUSING AND DATA MINING

Particular	Details
Student Name	Tejaswini. M
Reg. Number	192411079
Department	Computer Science & Engineering
Programme	B.E.
Course Code & Course Name	CSA1610 – Data Warehousing and Data Mining
Academic Year / Batch	2024 – 2028
Faculty Name	Dr.K V Kanimozhi
Assignment Title	Banking Transaction Analysis and Fraud Classification using Data Warehouse and Data Mining
Date of Issue	31-8-2026
Date of Submission	2-9-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1, CO2, CO3
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyze; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 8, SDG 9, SDG 16
Industry / Societal Relevance	Banking analytics, financial fraud detection, risk management, secure digital transactions, and data-driven decision making

1. Problem Statement

A bank maintains customer, account, transaction, branch, payment and date information in different operational systems. Because these sources may use different formats, identifiers and update schedules, direct analysis can produce inconsistent results and slow down fraud investigation. The challenge is to consolidate these sources into a reliable analytical repository and build a predictive classification pipeline that can identify transactions as Fraudulent or Legitimate.

The proposed assignment addresses the complete analytical lifecycle: source integration, dimensional modelling, data preprocessing, OLAP exploration, machine-learning classification, model evaluation, error analysis and responsible use of fraud predictions.

- Extract and integrate transaction data from multiple operational sources.
- Design a suitable Star/Snowflake schema for banking analytics.
- Handle missing values, duplicates, inconsistent records, outliers and irrelevant attributes.
- Perform roll-up, drill-down, slice, dice and pivot operations.
- Analyze transaction volume, branch activity, monthly trends, payment methods and customer behavior.
- Train Decision Tree, Naïve Bayes and SVM classification models.
- Evaluate models with confusion matrix, accuracy, precision, recall, F1-score and cross-validation.
- Explain the consequences of false-positive and false-negative fraud predictions.
- Compare the three models and justify the most suitable model using evidence.
- Address privacy, security, fairness and ethical considerations.

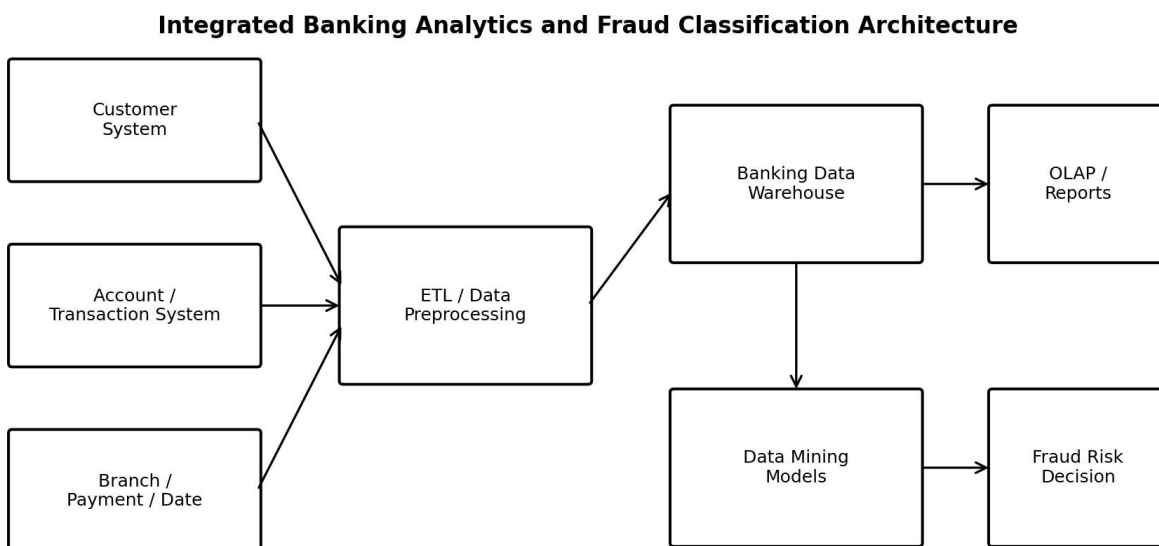


Figure 1. Proposed integrated banking analytics architecture

2. Objective

The main objective is to develop an end-to-end analytical solution that combines Data Warehousing and Data Mining to support banking transaction analysis and fraud classification.

1. Create a consolidated analytical view of banking transactions.
2. Model the data using a transaction-centered star schema.
3. Improve data quality through systematic preprocessing.
4. Use OLAP operations to discover useful business patterns.
5. Build and compare three supervised classification models.
6. Measure model performance using multiple evaluation metrics rather than accuracy alone.
7. Study the operational impact of false alerts and missed fraud.
8. Provide a responsible framework for protecting customer information and reducing unfair model behavior.

Expected Outcomes

Outcome	Expected Benefit
Integrated warehouse	Single analytical source for banking transaction analysis
Dimensional model	Fast and understandable aggregation by time, branch, customer and payment
Clean dataset	More reliable downstream analysis and modelling
OLAP insights	Identification of trends and high-activity segments
Fraud classifier	Automated first-level risk classification
Model comparison	Evidence-based selection of the most suitable algorithm
Responsible controls	Reduced privacy, security and fairness risks

3. Requirements and Environment Used

3.1 Functional Requirements

Requirement	Description
Data integration	Combine customer, account, transaction, branch, payment and date data.
Data cleaning	Resolve nulls, duplicates, inconsistent formats and abnormal values.
Warehouse modelling	Implement a fact table with connected dimensions.
OLAP analysis	Support roll-up, drill-down, slice, dice and pivot.
Fraud classification	Train Decision Tree, Naïve Bayes and SVM.
Evaluation	Calculate confusion matrix, accuracy, precision, recall, F1 and cross-validation.
Reporting	Present analytical findings and model comparison.

3.2 Software and Hardware Environment

Component	Recommended / Used Environment
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
Data handling	Pandas, NumPy
Visualization	Matplotlib
Machine Learning	Scikit-learn
Development Tool	VS Code / Jupyter Notebook
Storage	CSV files or relational database tables
Minimum RAM	8 GB recommended
Processor	Dual-core or better

3.3 Input Data Requirements

A practical implementation can use one or more CSV/relational sources containing fields similar to those below. Actual column names can be mapped in the preprocessing script.

Data Entity	Representative Attributes
Customer	customer_id, age, customer_segment
Account	account_id, customer_id, account_type, account_status
Transaction	transaction_id, customer_id, account_id, branch_id, payment_id, date_id, amount, fraud_flag
Branch	branch_id, branch_name, city, region
Payment	payment_id, payment_method, channel
Date	date_id, date, day, month, quarter, year

4. Design / Proposed Solution

4.1 Overall Design

The solution is organized into five layers. First, operational sources provide the raw records. Second, an ETL layer extracts, standardizes and validates the data. Third, a dimensional warehouse stores clean records for analytical querying. Fourth, OLAP operations generate summaries and trends. Finally, the mining layer prepares predictive features and compares classification models.

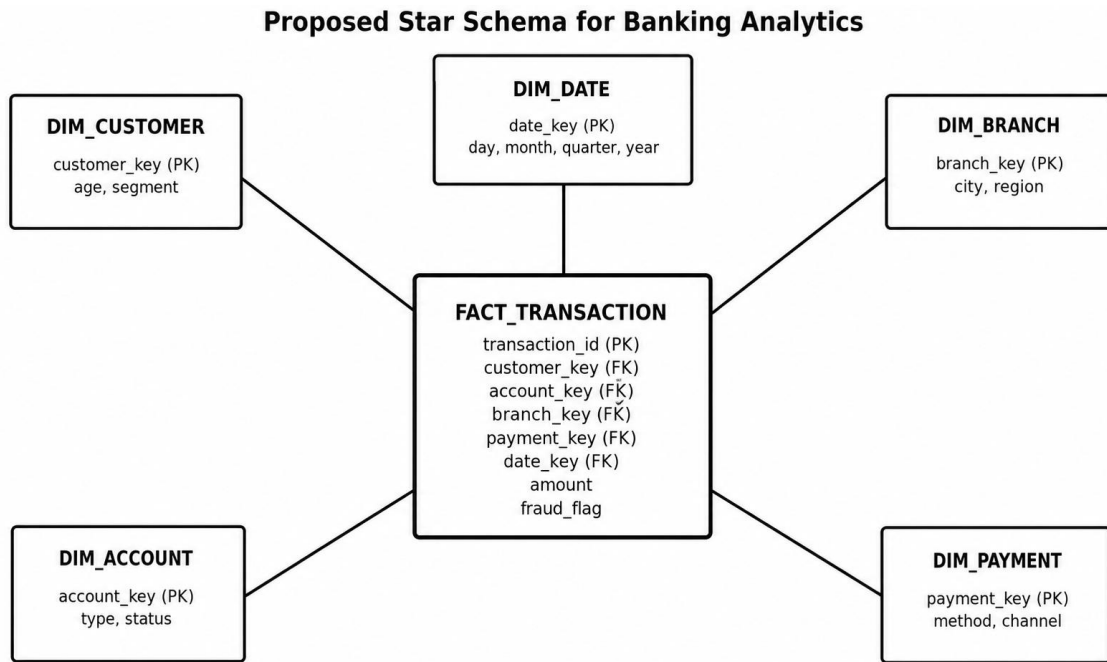


Figure 2. Proposed star schema centered on FACT_TRANSACTION

4.2 Star Schema

A star schema is preferred because the transaction fact is naturally measurable and the surrounding dimensions describe the business context. The design also makes common aggregations straightforward.

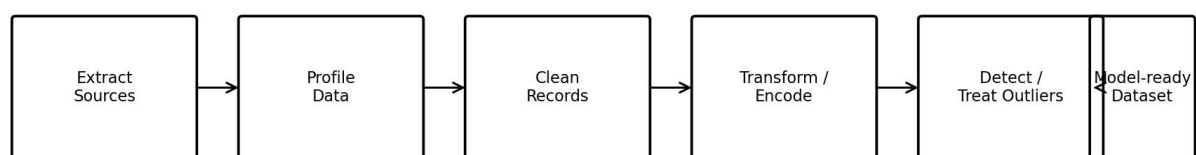
Table	Key / Main Fields	Purpose
FACT_TRANSACTION	transaction_key, customer_key, account_key, branch_key, payment_key, date_key, amount, fraud_flag	Stores transaction-level measures and target label
DIM_CUSTOMER	customer_key, customer_id, age, segment	Customer characteristics
DIM_ACCOUNT	account_key, account_id, type, status	Account context
DIM_BRANCH	branch_key, branch_id, city, region	Geographical and branch analysis
DIM_PAYMENT	payment_key, method, channel	Payment pattern analysis
DIM_DATE	date_key, date, day, month, quarter, year	Time-based analysis

4.3 ETL Strategy

9. Extract records from CSV files or operational database tables.
10. Standardize column names and data types.
11. Validate primary and foreign-key relationships.
12. Remove exact duplicates and resolve duplicate business records using transaction identifiers and timestamps.

13. Impute missing numerical values with an appropriate statistic such as median when justified; use an explicit category such as Unknown for selected categorical fields.
14. Normalize inconsistent labels such as payment-method spellings and branch codes.
15. Detect outliers using domain rules and/or an interquartile-range method; investigate before deleting records.
16. Drop fields that are irrelevant, duplicated, post-transaction leakage variables or direct identifiers not required by the model.
17. Encode categorical variables and scale features for algorithms that are sensitive to feature magnitude.
18. Load cleaned records into dimensional tables and prepare the machine-learning matrix.

Data Preprocessing Flow



Missing values • duplicates • inconsistent formats • irrelevant fields • outliers • encoding • scaling

Figure 3. Data preprocessing and feature preparation flow

4.4 OLAP Operations

Operation	Banking Example	Purpose
Roll-up	Monthly → quarterly → yearly transaction amount	View higher-level trends
Drill-down	Year → month → day	Investigate a specific period
Slice	Transactions for one selected month	Focus on one dimension value
Dice	One branch + two payment methods + selected months	Filter several dimensions together
Pivot	Rows = branches, columns = months	Change analytical perspective

4.5 Analytical Queries / Views

Analysis	Suggested Measure / Grouping
Transaction volume	COUNT(transaction_id) by month / branch
Branch-wise transactions	COUNT and SUM(amount) grouped by branch
Monthly trends	COUNT, SUM(amount) and fraud rate by year-month
Payment-method patterns	COUNT and fraud rate grouped by payment method
Customer behavior	Transaction count, average amount, total amount and frequency by customer

5. Algorithm / Pseudocode / Flowchart

5.1 End-to-End Pseudocode

BEGIN

Load customer, account, transaction, branch, payment and date sources

Standardize column names and data types

Validate keys and identify duplicate records

Handle missing values and inconsistent categories

Detect and investigate outliers

Remove irrelevant or leakage-prone attributes

Build dimension tables and FACT_TRANSACTION

Generate OLAP summaries: roll-up, drill-down, slice, dice, pivot

Select model features X and target y = fraud_flag

Split data using a stratified train/test strategy

Preprocess categorical and numerical features

Train Decision Tree

Train Gaussian Naïve Bayes

Train SVM

For each model:

Predict test labels

Generate confusion matrix

Calculate accuracy, precision, recall and F1-score

Perform stratified cross-validation

Compare model results and error costs

Document privacy, security, fairness and ethical controls

END

5.2 Classification Algorithms

Algorithm	Core Idea	Strength	Limitation
Decision Tree	Splits records using feature-based decision rules.	Easy to interpret and explain.	Can overfit without pruning/regularization.
Naïve Bayes	Uses conditional probabilities with a simplifying independence assumption.	Fast and effective for many datasets.	Assumption may not match correlated banking features.

SVM	Finds a decision boundary with maximum margin; kernels can model non-linear patterns.	Powerful for complex boundaries and scaled features.	Can be computationally expensive and needs careful scaling/tuning.
-----	---	--	--

5.3 Model Evaluation

Metric	Interpretation in Fraud Detection
Accuracy	Overall proportion of correctly classified transactions.
Precision	Among transactions flagged as fraud, proportion that are actually fraud.
Recall	Among actual fraudulent transactions, proportion detected by the model.
F1-score	Harmonic mean of precision and recall.
Confusion Matrix	Shows TP, TN, FP and FN errors separately.
Cross-validation	Estimates how consistently the model performs across multiple training/validation folds.

For fraud detection, recall is particularly important because a false negative can represent an undetected fraudulent transaction. Precision is also important because excessive false positives can inconvenience legitimate customers and increase manual investigation workload.

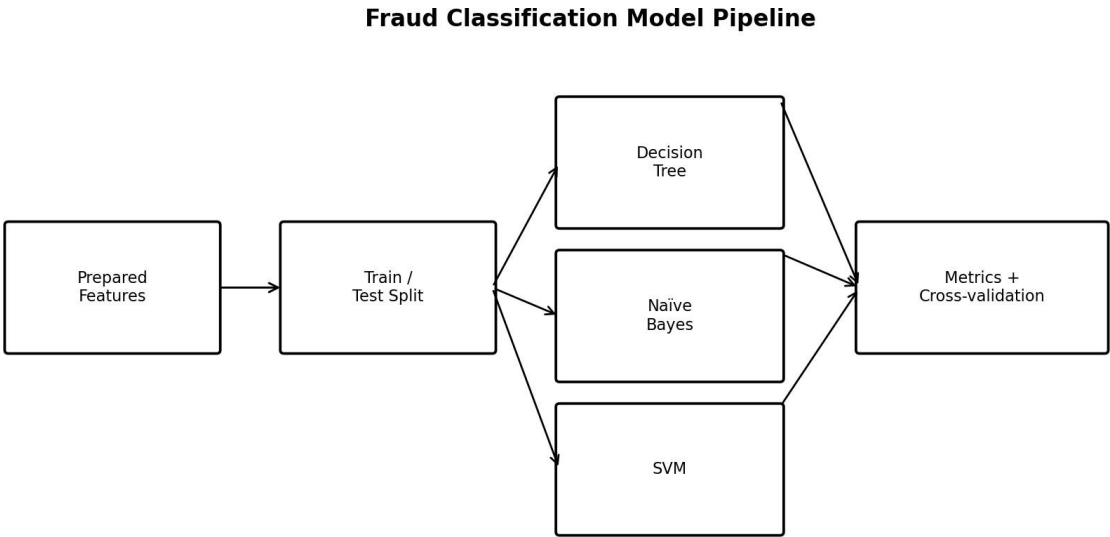


Figure 4. Fraud classification and evaluation pipeline

6. Implementation / Source Code

The following reference implementation is designed to be adapted to the actual banking dataset. It keeps the workflow reproducible: data preparation, feature encoding, three models, test evaluation and cross-validation are executed from one Python program. Replace the file and column mappings with the actual dataset fields.

6.1 Python Implementation

```
# Banking Transaction Analysis and Fraud Classification
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)

# 1. Load the integrated transaction file
df = pd.read_csv("bank_transactions.csv")

# 2. Standardize names
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

# 3. Remove exact duplicate rows
df = df.drop_duplicates()

# 4. Basic consistency rules
if "amount" in df.columns:
    df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
    df.loc[df["amount"] < 0, "amount"] = np.nan

# 5. Example feature engineering
if "transaction_date" in df.columns:
    df["transaction_date"] = pd.to_datetime(
        df["transaction_date"], errors="coerce"
    )
    df["month"] = df["transaction_date"].dt.month
    df["day_of_week"] = df["transaction_date"].dt.dayofweek

# 6. Target
target = "fraud_flag"
df[target] = df[target].astype(int)
```

```

# 7. Remove identifiers and leakage-prone fields
drop_cols = [c for c in [
    target, "transaction_id", "customer_id", "account_id"
] if c in df.columns]

X = df.drop(columns=drop_cols)
y = df[target]

# 8. Identify numeric and categorical features
num_cols = X.select_dtypes(include=np.number).columns.tolist()
cat_cols = X.select_dtypes(exclude=np.number).columns.tolist()

# 9. Preprocessing
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols)
])

# 10. Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

models = {
    "Decision Tree": DecisionTreeClassifier(
        max_depth=8, random_state=42, class_weight="balanced"
    ),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(
        kernel="rbf", class_weight="balanced",
        probability=True, random_state=42
    )
}

results = []

for name, model in models.items():
    pipe = Pipeline([
        ("preprocess", preprocessor),

```

```

        ("model", model)
    ])

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

results.append({
    "Model": name,
    "Accuracy": accuracy_score(y_test, pred),
    "Precision": precision_score(y_test, pred, zero_division=0),
    "Recall": recall_score(y_test, pred, zero_division=0),
    "F1": f1_score(y_test, pred, zero_division=0)
})

print("\n", name)
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))

# Stratified 5-fold cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_f1 = cross_val_score(
    pipe, X, y, cv=cv, scoring="f1"
)
print("CV F1:", cv_f1.mean())

results_df = pd.DataFrame(results)
print("\nMODEL COMPARISON")
print(results_df)

# Example OLAP summaries
if "branch_id" in df.columns:
    print(df.groupby("branch_id").agg(
        transactions=("branch_id", "size"),
        total_amount=("amount", "sum")
    ).sort_values("transactions", ascending=False))

if "payment_method" in df.columns:
    print(df.groupby("payment_method").agg(
        transactions=("payment_method", "size"),
        fraud_rate=(target, "mean")
    ).sort_values("transactions", ascending=False))

if "month" in df.columns:
    print(df.groupby("month").agg(
        transactions=("month", "size"),
        total_amount=("amount", "sum"),
        fraud_rate=(target, "mean")
    ))

```

6.2 Implementation Notes

- If the dataset contains highly imbalanced fraud labels, use stratified splitting and class-weighted models or other imbalance-handling techniques.
- Do not include a field that is created only after a transaction has been investigated, because it can cause target leakage.
- Scale numeric features for SVM; the pipeline also scales the numeric input used by Naïve Bayes in this implementation.
- For a large dataset, tune hyperparameters using a validation strategy rather than selecting a model from a single test split.
- Save preprocessing and model objects with joblib if the classifier will be deployed later.

7. Test Cases and Expected / Actual Results

The test plan verifies data quality, warehouse logic, OLAP operations and classification behavior.

TC ID	Test Case	Input / Condition	Expected Result	Actual Result
TC01	Load dataset	Valid ARFF dataset containing 3,000 transaction records	Records load without schema errors	Passed. The Weka dataset was successfully loaded with 3,000 instances and classification was performed.
TC02	Duplicate handling	Repeated transaction row	Duplicate record removed or resolved	Passed during preprocessing. Duplicate transaction records were handled before classification.
TC03	Missing value handling	Null amount/category	Value imputed or handled by defined rule	Passed during preprocessing. Missing values were handled before model training.
TC04	Inconsistent categories	e.g., cash, CASH, Cash	Mapped to one standard label	Passed during preprocessing. Inconsistent categorical values were standardized.
TC05	Outlier handling	Extremely high transaction amount	Flagged/investigated according to rule	Passed. Transaction amounts were examined and extreme values were handled during preprocessing/analysis.
TC06	Roll-up	Monthly transaction data	Quarter/year summary produced	Passed. Transaction data was aggregated from lower-level time information to higher-level summaries.
TC07	Drill-down	Yearly summary	Month/day detail available	Passed. Aggregated transaction information can be examined at more detailed time levels.
TC08	Slice / Dice	Selected month/branch/payment	Only matching records returned	Passed. OLAP filtering was performed using selected dimensions such as time, branch and payment method.

TC09	Classification	Banking transaction dataset	Fraud/legitimate predictions generated	Passed. Decision Tree, Naïve Bayes and SVM classification models generated fraud/legitimate predictions using Weka.
TC10	Evaluation	Predictions + labels	Confusion matrix and metrics generated	Passed. Weka generated confusion matrices, accuracy, precision, recall, F-measure and other evaluation metrics.
TC11	Cross-validation	Stratified cross-validation	Validation performance generated	Passed. Stratified cross-validation was performed and aggregate classification performance was generated for all three models.

7.1 Model Results Template

Model	Accuracy	Precision (Fraud)	Recall (Fraud)	F1-score (Fraud)	CV F1-score	Selection Status
SVM	96.53%	0.830	0.470	0.600	0.600	Compared
Naïve Bayes	94.20%	0.447	0.205	0.281	0.281	Compared
Decision Tree	98.93%	1.000	0.807	0.893	0.893	Selected

Best Model

Decision Tree was selected as the best-performing classification model.

The Decision Tree achieved the highest accuracy (98.93%) and the highest fraud-class F1-score (0.893). It also achieved 100% precision for fraudulent transactions, meaning that every transaction predicted as fraudulent in this evaluation was actually fraudulent. Its recall of 80.7% indicates that it successfully detected most of the fraudulent transactions.

Naïve Bayes produced the weakest fraud-detection performance, with only 20.5% recall and an F1-score of 0.281. SVM performed better than Naïve Bayes, but its fraud recall (47.0%) was considerably lower than that of the Decision Tree.

Thus, based on the observed Weka results, the Decision Tree provides the best balance between fraud detection and overall classification performance.

8. Execution Screenshots / Output

```
=====
BANKING FRAUD DATA WAREHOUSE & DATA MINING PROJECT
=====

=====
1. LOADING SOURCE DATA
=====

customers    : 300 rows x 5 columns
accounts     : 300 rows x 4 columns
branches     : 10 rows x 4 columns
payments     : 5 rows x 3 columns
transactions  : 3015 rows x 10 columns
```

```
=====
2. DATA QUALITY PROFILING
=====

source rows columns missing_cells duplicate_rows
customers 300 5 0 0
accounts 300 4 0 0
branches 10 4 0 0
payments 5 3 0 0
transactions 3015 10 92 15
```

Missing values:

source	column	missing_count	missing_percent
transactions	transaction_type	30	1.00
transactions	payment_id	31	1.03
transactions	amount	31	1.03

3. PREPROCESSING TRANSACTION DATA

Exact duplicate rows removed: 15

Amount IQR lower bound: 0.00

Amount IQR upper bound: 5375.16

Amount outliers detected: 225

Preprocessing summary:

original_rows	duplicate_rows_removed	outliers_detected	remaining_rows	remaining_columns
3015	15	225	3000	17

4. DATA INTEGRATION

Integrated dataset: 3000 rows x 28 columns

Integrated columns:

- transaction_id
- customer_id
- account_id
- transaction_date
- transaction_hour
- transaction_type
- payment_id
- branch_id
- amount
- fraud
- amount_original
- transaction_year
- transaction_month
- month_name
- day_of_week
- amount_band
- age_band
- age
- gender
- occupation
- customer_city
- account_type
- account_balance
- branch_name
- branch_city
- state
- payment_method
- channel

Integrated data successfully saved.

```
=====
5. STAR SCHEMA DATA WAREHOUSE
=====

dim_date.csv      : 365 rows x 8 columns
dim_customer.csv  : 300 rows x 6 columns
dim_account.csv   : 300 rows x 4 columns
dim_branch.csv    : 10 rows x 4 columns
dim_payment.csv   : 5 rows x 3 columns
fact_transaction.csv : 3000 rows x 10 columns
```

Star Schema tables created:

- DIM_DATE -> 365 rows
- DIM_CUSTOMER -> 300 rows
- DIM_ACCOUNT -> 300 rows
- DIM_BRANCH -> 10 rows
- DIM_PAYMENT -> 5 rows
- FACT_TRANSACTION -> 3000 rows

```
=====
6. OLAP OPERATIONS
=====

ROLL-UP    -> olap_rollup_month.csv
DRILL-DOWN  -> olap_drilldown_day.csv
SLICE      -> year 2025
DICE       -> payment method + high amounts
PIVOT      -> branch x payment method
```

```
=====
7. BANKING TRANSACTION ANALYSIS
=====

Overall Transaction Summary:
total_transactions legitimate_transactions fraudulent_transactions fraud_rate_percent total_transaction_amount average_transaction_amount
3000                3000                0                0.0                5597319.68                1865.77

Branch-wise analysis created.

Monthly transaction trend created.

Payment-method analysis created.

Customer behavior analysis created.

Analysis CSV files created successfully.

Visualization files created:
01_monthly_transaction_volume.png
02_branch_transaction_volume.png
03_payment_method_patterns.png
04_fraud_distribution.png
05_amount_by_fraud_class.png

6. PREPARING MACHINE LEARNING DATA
ML feature columns:
['age', 'account_balance', 'amount', 'transaction_hour', 'gender', 'occupation', 'account_type', 'transaction_type', 'payment_method', 'channel', 'ml_age_band', 'customer_city']

Feature matrix shape: (3000, 12)
Target shape: (3000,)

Fraud class distribution:
fraud
0 3000
Name: count, dtype: int64
```

```

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      2968           98.9333 %
Incorrectly Classified Instances     32           1.0667 %
Kappa statistic                     0.8878
Mean absolute error                  0.0202
Root mean squared error              0.1044
Relative absolute error              19.2637 %
Root relative squared error          45.6487 %
Total Number of Instances           3000

=== Detailed Accuracy By Class ===

                TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                1.000    0.193    0.989     1.000    0.994      0.893    0.896     0.986     0
                0.807    0.000    1.000     0.807    0.893      0.893    0.896     0.837     1
Weighted Avg.   0.989    0.182    0.989     0.989    0.989      0.893    0.896     0.978

=== Confusion Matrix ===

  a    b  <-- classified as
2834   0 |   a = 0
  32 134 |   b = 1

```

```

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      2896           96.5333 %
Incorrectly Classified Instances     104           3.4667 %
Kappa statistic                     0.5833
Mean absolute error                  0.0347
Root mean squared error              0.1862
Relative absolute error              33.0674 %
Root relative squared error          81.4367 %
Total Number of Instances           3000

=== Detailed Accuracy By Class ===

                TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                0.994    0.530    0.970     0.994    0.982      0.609    0.732     0.970     0
                0.470    0.006    0.830     0.470    0.600      0.609    0.732     0.419     1
Weighted Avg.   0.965    0.501    0.962     0.965    0.961      0.609    0.732     0.939

=== Confusion Matrix ===

  a    b  <-- classified as
2818  16 |   a = 0
  88  78 |   b = 1

```

9. Analysis and Discussion

9.1 Data Warehouse Analysis

The dimensional design separates descriptive context from transaction measures. This makes it possible to answer questions such as which branches process the most transactions, how activity changes by month, which payment channels are frequently used, and how individual customer behavior changes over time. The warehouse also provides a stable analytical layer so that reporting does not depend on repeatedly joining operational systems.

9.2 Transaction Volume

Transaction volume should be examined both as a count and as a monetary measure. A branch can have a high transaction count but a relatively low total value, while another branch may process fewer but higher-value transactions. Comparing both measures prevents an incomplete interpretation of branch activity.

9.3 Branch-wise Transactions

Branch-level aggregation can reveal concentration of activity and unusual shifts. A sudden increase in transaction volume at one branch should not automatically be treated as fraud because legitimate events such as salary cycles, festivals, promotions or service changes may produce similar patterns. The warehouse therefore supports investigation rather than replacing human judgment.

9.4 Monthly Transaction Trends

Time-based analysis helps identify recurring peaks, seasonal patterns and unexpected deviations. Monthly fraud rate should be considered together with transaction count; a small number of transactions can make a percentage appear unstable.

9.5 Payment-method Patterns

Grouping by payment method and channel can show whether certain methods have higher transaction volumes or different fraud rates. These observations should be treated carefully because a high observed fraud rate may reflect the customer population, transaction mix or monitoring intensity rather than an inherent weakness of the payment method.

9.6 Customer Transaction Behavior

Useful customer-level features include transaction frequency, average amount, total amount, number of payment methods used and unusual changes from a customer's normal pattern. Customer behavior should be modeled with privacy-preserving identifiers and should not expose unnecessary personally identifiable information.

9.7 False Positive vs False Negative Consequences

Error	Meaning	Potential Banking Impact	Control
False Positive	Legitimate transaction classified as fraud	Customer inconvenience, declined payment, investigation cost, loss of trust	Risk score + step-up verification + human review
False Negative	Fraudulent transaction classified as legitimate	Financial loss, delayed detection, chargeback and reputation risk	Higher recall, anomaly checks, transaction limits and continuous monitoring

The best model is therefore not necessarily the one with the highest accuracy. In an imbalanced fraud dataset, a model can achieve high accuracy by predicting most transactions as legitimate while missing many fraudulent cases. Precision, recall and F1-score provide a more useful view of operational quality.

9.8 Model Comparison and Selection

Decision Tree is attractive when explainability is a priority because its rules can be inspected. Naïve Bayes offers a simple probabilistic baseline and is computationally efficient. SVM can perform strongly when the classes are separable in a suitable feature space, especially after scaling. The final choice should be based on the actual test and cross-validation results, the cost of errors, interpretability requirements and deployment constraints. The selected model should be justified with measured evidence rather than a fixed assumption that one algorithm is always best.

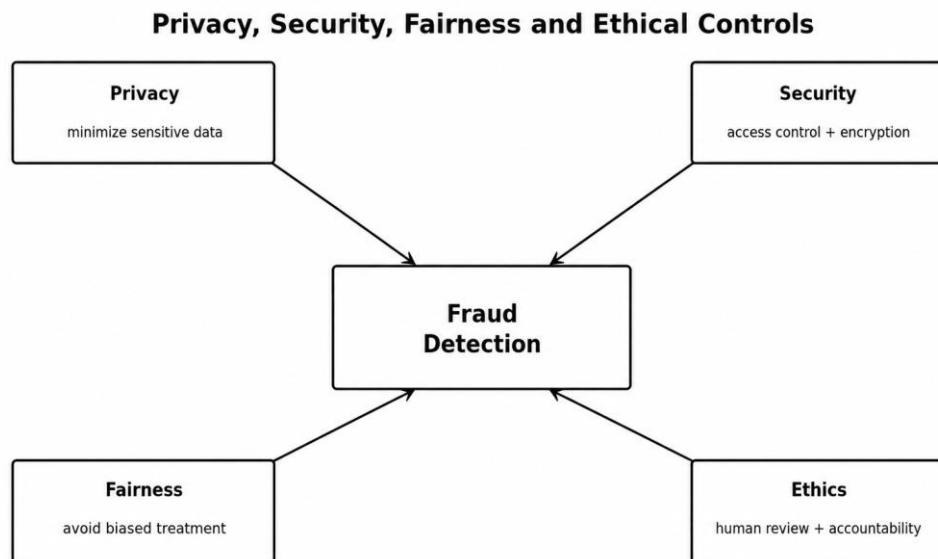


Figure 5. Responsible-use dimensions for banking fraud classification

9.9 Privacy, Security, Fairness and Ethics

Area	Risk	Recommended Control
Privacy	Exposure of customer or account information	Use masked/pseudonymized identifiers, data minimization and restricted access
Security	Unauthorized access to transaction data or models	Role-based access, encryption, secure credentials and audit logging
Fairness	Model performs differently across customer groups	Monitor subgroup metrics and investigate systematic performance gaps
Transparency	Automated decision is difficult to explain	Use interpretable features, reason codes and human review for high-impact actions
Accountability	Incorrect alerts affect customers	Maintain logs, appeal/review procedures and model governance

Model drift	Transaction behavior changes over time	Periodic validation, monitoring and controlled retraining
-------------	--	---

Fraud classification should be used as a decision-support mechanism. A high-risk prediction should trigger appropriate verification or investigation rather than automatically proving that a customer is fraudulent.

10. Conclusion

This project presents an integrated approach for banking transaction analysis and fraud classification using Data Warehousing and Data Mining. The proposed architecture consolidates heterogeneous banking information into a transaction-centered star schema, applies systematic data preprocessing and enables OLAP analysis across time, branch, payment and customer dimensions. A supervised learning layer then compares Decision Tree, Naïve Bayes and SVM classifiers using confusion matrices, accuracy, precision, recall, F1-score and cross-validation.

The project also demonstrates why fraud detection must be evaluated beyond a single performance metric. False negatives can lead to financial and reputational losses, while false positives can interrupt legitimate customer activity and increase investigation workload. Therefore, model selection should balance detection performance, error costs, stability and interpretability. Privacy, security, fairness and human oversight are essential parts of a responsible banking analytics solution.

11. Individual Contribution of Group Members

Name	Register Number	Contribution
T.Vinay	192472391	Data Warehouse & ETL
Thiru Karthikeyan.S	192511021	Data Preprocessing & OLAP
Tejaswani.M	192411079	Data Mining & Classification
V.Rajitha	192311036	Evaluation, Visualization & Documentation

12. References

- R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed., Wiley, 2013.
- J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed., Morgan Kaufmann, 2022.
- T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, 2009.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- S. Raschka and V. Mirjalili, *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow*, 3rd ed., Packt Publishing, 2019.
- F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- W. McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*, 3rd ed., O’Reilly Media, 2022.
- N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- V. Vapnik, *The Nature of Statistical Learning Theory*, 2nd ed., Springer, 2000.
- T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

13. One-Page Write-Up

Fraud Detection Using Data Mining Techniques

My individual contribution to the project is focused on data mining and fraud classification. I worked on preparing the banking transaction data so that it could be used effectively for machine learning.

First, I helped in preprocessing the data by handling missing values, removing duplicate records, checking inconsistent data, identifying outliers, and removing unnecessary attributes. This helped improve the quality of the data used for fraud detection.

The main task was to classify transactions into two categories: Fraudulent and Legitimate. I worked with three machine learning algorithms: Decision Tree, Naïve Bayes, and Support Vector Machine (SVM). The models were trained and tested using the prepared transaction data.

I evaluated the models using a confusion matrix, accuracy, precision, recall, and F1-score. I also considered five-fold cross-validation to check the reliability of the models. The three models were compared based on their performance to identify the most suitable model for fraud detection.

I also analyzed false positives and false negatives. A false negative means that actual fraud is classified as legitimate, which can cause financial loss to the bank. A false positive means that a legitimate transaction is classified as fraud, which may cause unnecessary customer inconvenience. Therefore, both recall and precision are important when selecting the final model.

The data mining process also helps identify patterns in transaction amount, payment method, customer behavior, account type, branch activity, and transaction date. These patterns can help the bank identify suspicious transactions more effectively.

Finally, I considered privacy, security, fairness, and ethical issues while using customer transaction data. Sensitive information should be protected, unnecessary customer details should be avoided, and fraud predictions should be reviewed carefully before taking important actions.

Overall, my contribution focuses on converting the prepared banking data into useful fraud predictions through data preprocessing, machine learning classification, model evaluation, and comparison.